CARRERA DE: DESARROLLO DE SOFTWARE

NIVEL: PRIMERO

MEMORIA DEL PROYECTO ACADÉMICO DE INNOVACIÓN TECNOLÓGICA

TÍTULO:

Resolución de Problemas Matemáticos

AUTORES:

- Adrian Campos
- Dennis Yaguache
- Oscar Manobanda
- Ismael Venegas

TUTOR: Msgtr. Estefani Sanchez

QUITO-ECUADOR

2025

INDICE

1. RESUMEN 4
2. INTRODUCCIÓN: 5
3. OBJETIVO GENERAL 8
4. MARCO TEÓRICO 9
4.1. Fundamentos de programación y algoritmos 9
4.1.1. ¿Qué es la programación? 9
4.1.2. ¿Qué es un algoritmo? 10
4.1.3. ¿Qué es el pseudocódigo? 10
4.2. INTRODUCCIÓN A JAVA – COMPILACIÓN Y EJECUCIÓN 11
4.2.1. ¿Qué es Java? 11
4.2.2. ¿Qué es la compilación y ejecución? 11
4.3. INSTALACIÓN DEL JDK, IDE Y HOLA MUNDO 12
4.3.1. ¿Qué es el JDK? 12
4.3.2. ¿Qué es un IDE? 13
4.4. Operadores y estructuras condicionales 14
4.4.1. ¿Qué son los operadores aritméticos? 14
4.4.2. ¿Qué es la estructura if–else? 15
4.4.3. ¿Qué es el operador ternario? 16
4.5. Modularización 16
4.5.1. Métodos de uso 17
4.5.2. Modularización en Variable 18
4.6. Sobrecarga de Métodos (scope) 19
4.6.1. Componentes clave 19
4.7. Arreglos Unidimensionales 20
4.7.1. Declaración 21
4.8. Recorrido y Búsqueda de arreglos 22
4.8.1. Recorridos de Arreglos 22
4.9. Método de Recorrido: Bucle for 22
4.9.1. Bucle for-each 23
4.10. Búsqueda de Arreglos 23
4.10.1. Búsqueda Lineal 23
4.10.2. Búsqueda Binaria 23
4.11. Ordenamiento Básico (Burbuja) 24
4.11.1. Cómo Funciona 24
4.12. Arreglos Multidimensionales 25

4.13. Arreglo Bidimensional (Matriz) 25
4.14. Matrices y su Manipulación 26
4.15. Operaciones comunes de manipulación de matrices 26
4.15.1. Multiplicación de Matrices 26
4.15.2. Trasposición de Matriz 26
4.16. Cadenas de caracteres (String) 27
4.16.1. Características Clave de las Cadenas de Caracteres 28
5. MATEMÁTICA 29
5.1. Trinomio cuadrado perfecto 29
5.1.1. Trinomio de la forma ax2 + bx +c 29
5.1.2. Suma de cubos 30
5.2. Diferencia de cubos 30
5.3. Ecuaciones de primer grado 31
5.3.1. Lenguaje algebraico 31
5.3.2. Definición ecuación lineal simple 31
5.4. Método de Resolución de Sistema de Ecuaciones Lineales 32
5.4.1. Método por determinantes 32
5.4.2. Método grafico 33
5.5. Ecuaciones de 2do grado 34
5.6. Resolución por Factorización. 35
5.7. Rango de una función 36
5.8. Intervalos de positividad y negatividad 36
5.9. Monotonía de la Función 37
5.10. Asíntota verticales y horizontales 37
5.10.1. Asíntotas Verticales 37
5.11. Función Cuadrática 38
5.11.1. Definición y Representación 38
5.12. Concavidad de una Función Cuadrática 38
5.13. Vértice de la Función Cuadrática 38
5.14. Raíces reales de una Función 39
6. RESULTADOS 39
7. BIBLIOGRAFÍA 44

1. RESUMEN

Este proyecto académico, desarrollado por estudiantes de la carrera de Desarrollo de Software del Instituto Superior Tecnológico Mayor "Pedro Traversari", propone una solución tecnológica integral ante la fragmentación del aprendizaje entre las ciencias de la computación y las matemáticas. El documento describe el diseño e implementación de un sistema informático avanzado en lenguaje Java, dotado de una interfaz gráfica de usuario (GUI) interactiva, cuyo propósito central es resolver problemas matemáticos de nivel intermedio y avanzado que abarcan desde ecuaciones lineales hasta el manejo complejo de matrices y funciones.

La investigación parte de una problemática detectada en la educación técnica superior: la desconexión existente entre la teoría matemática abstracta y su aplicación práctica en el desarrollo de software. Los estudiantes suelen resolver ejercicios en papel que nunca llegan a ser implementados computacionalmente, lo que debilita el pensamiento algorítmico. Para cerrar esta brecha, el sistema desarrollado utiliza la robustez y portabilidad de Java para materializar conceptos como el trinomio cuadrado perfecto, la diferencia de cubos y sistemas de ecuaciones mediante métodos de factorización, determinantes (regla de Cramer) y fórmulas generales.

Desde una perspectiva técnica, el software integra componentes clave del sílabo de programación, incluyendo estructuras de control "if-else", operadores aritméticos, modularización y sobrecarga de métodos. Además, implementa el uso de arreglos unidimensionales y multidimensionales (matrices) para organizar datos en forma de cuadrículas, facilitando operaciones como la suma, multiplicación y trasposición de matrices. Un aspecto innovador del proyecto es la inclusión de persistencia de datos mediante las librerías "FileReader" y "FileWriter",

lo que permite mantener un registro histórico de las operaciones realizadas y generar reportes para el usuario.

Los resultados finales muestran una "Calculadora Master" con múltiples ventanas dedicadas a la resolución paso a paso de ecuaciones de primer y segundo grado, así como el cálculo de áreas y perímetros de figuras geométricas como cuadrados, triángulos, rectángulos y círculos. Más allá de ser una herramienta funcional, el proyecto se erige como una innovación educativa de código abierto que democratiza el acceso a recursos tecnológicos de calidad, fortaleciendo habilidades blandas como el trabajo colaborativo y la resolución de problemas en un entorno profesional real.

2. INTRODUCCIÓN:

En el ecosistema tecnológico contemporáneo, la intersección entre las ciencias de la computación y las matemáticas ha dejado de ser una mera relación académica para convertirse en la piedra angular del desarrollo societal. Como señalan Fernández, Riveros y Montiel, las Tecnologías de Información y Comunicación (TIC), y específicamente el software matemático, han conquistado el accionar humano, adquiriendo un rol protagónico que facilita la llegada de innovaciones a todos los sectores de la sociedad. Esta simbiosis es especialmente crítica en el ámbito educativo, donde el software no solo actúa como un medio de comunicación e información, sino como un entorno interactivo que permite construir conocimiento de manera culturalmente esencial.

La incorporación de herramientas digitales en el estudio de las matemáticas permite recrear situaciones de la realidad del estudiante, logrando que conceptos abstractos se vuelvan tangibles y significativos. En este sentido, lenguajes de programación de alto nivel como Java se posicionan

como instrumentos fundamentales debido a su robustez, portabilidad y amplia adopción tanto en la academia como en la industria. Java ofrece un ecosistema propicio para materializar algoritmos complejos mediante un sistema de tipos estático y una estructura orientada a objetos que garantiza soluciones escalables y eficientes.

A pesar de la potencia de estas herramientas, la educación técnica superior enfrenta un desafío persistente: la fragmentación del aprendizaje. En el contexto de la carrera de Tecnología Superior en Desarrollo de Software, se observa una desconexión significativa entre la teoría de los fundamentos de programación y su aplicación práctica en la resolución de problemas matemáticos integrales. Los estudiantes del primer nivel, aunque cursan simultáneamente asignaturas como Fundamentos de Programación y Matemática, suelen experimentar una limitada transferencia de conocimientos entre ambas áreas.

Esta problemática se manifiesta en la dificultad para transformar conceptos matemáticos —tales como sistemas de ecuaciones, funciones cuadráticas o geometría analítica— en código ejecutable y funcional. El modelo pedagógico tradicional a menudo lleva a que los alumnos programen algoritmos aislados en Java sin comprender su utilidad práctica, mientras resuelven ejercicios matemáticos en papel que nunca llegan a ser implementados computacionalmente. Además, existe una escasa disponibilidad de software educativo desarrollado localmente que permita visualizar el proceso de resolución paso a paso, ya que las aplicaciones comerciales suelen centrarse en el resultado final y no en el valor pedagógico del algoritmo subyacente. La falta de proyectos integradores resulta, por tanto, en un pensamiento algorítmico debilitado y en una percepción de la matemática como un área abstracta, aburrida y desconectada de la cotidianidad profesional.

La implementación de este proyecto se justifica desde múltiples dimensiones. Académicamente, busca cerrar la brecha entre la teoría y la praxis, respondiendo a la necesidad de crear herramientas que permitan a los futuros desarrolladores visualizar algoritmos de manera concreta. La evidencia en educación de ingeniería sugiere que los proyectos interdisciplinarios mejoran drásticamente la retención de conocimientos y consolidan un pensamiento computacional sólido.

Desde la perspectiva tecnológica, el uso de Java con interfaces gráficas (GUI) mediante librerías como Swing o JavaFX dota al estudiante de competencias altamente demandadas en el mercado laboral. El desarrollo de este sistema permite aplicar de forma real conceptos de programación orientada a objetos, manejo de excepciones para la validación de datos y persistencia mediante archivos para el historial de operaciones. Finalmente, el proyecto posee un impacto social relevante al proponer una alternativa de código abierto que democratiza el acceso a recursos educativos de calidad, reduciendo la dependencia de software comercial costoso y beneficiando potencialmente a diversos niveles del sistema educativo.

El objetivo principal de esta iniciativa es desarrollar un sistema integral en Java con interfaz gráfica que resuelva problemas matemáticos de nivel intermedio-avanzado. Se busca integrar de manera coherente los contenidos del sílabo, incluyendo estructuras de control, métodos, arreglos multidimensionales y manejo de archivos, para fortalecer el aprendizaje significativo y el razonamiento lógico-matemático de los estudiantes.

Resolución de ecuaciones de primer y segundo grado (mediante fórmula general y factorización), así como sistemas de ecuaciones lineales por métodos como determinantes.

Análisis de funciones cuadráticas (vértices, ejes de simetría, concavidad) y cálculo de áreas y perímetros de figuras geométricas.

Implementación de persistencia de datos mediante FileReader y FileWriter, permitiendo generar reportes y mantener un registro de las operaciones realizadas.

Creación de un entorno interactivo y usable que incluya manejo de excepciones para garantizar la estabilidad del sistema ante entradas erróneas del usuario.

El presente documento se encuentra organizado de la siguiente manera: tras esta introducción, se detallan los antecedentes y el planteamiento del problema que fundamentan la investigación. Posteriormente, se exponen los objetivos y la justificación técnica y social. Se continúa con la descripción de los saberes y contenidos curriculares integrados, seguidos por la metodología de desarrollo empleada y el cronograma de actividades. Finalmente, se presentan las estrategias para el desarrollo de habilidades blandas, la bibliografía consultada y las rúbricas que rigen la evaluación de la innovación tecnológica propuesta.

3. OBJETIVO GENERAL

Desarrollar e implementar un sistema informático integral mediante el lenguaje de programación Java, dotado de una interfaz gráfica de usuario (GUI) interactiva y robusta, con el fin de resolver problemas matemáticos de nivel intermedio y avanzado que abarquen desde ecuaciones lineales y cuadráticas hasta el manejo complejo de matrices y funciones (pp. 1, 8). Este proyecto busca erigirse como un puente tecnológico y pedagógico que logre la convergencia efectiva entre los fundamentos teóricos de la programación orientada a objetos y el razonamiento lógico-matemático, superando la fragmentación del aprendizaje que actualmente afecta a los estudiantes de primer nivel de la carrera de Tecnología Superior en Desarrollo de Software.

A través de una arquitectura de software que priorice la escalabilidad y la eficiencia, el sistema integrará funcionalidades avanzadas como la persistencia de datos mediante el manejo de archivos (FileReader y FileWriter) para el registro histórico de operaciones, y un sólido sistema de gestión de excepciones que garantice la estabilidad ante errores de entrada (pp. 3, 6). Asimismo, el proyecto se propone fortalecer las competencias técnicas y las habilidades blandas de los estudiantes, tales como el pensamiento algorítmico, la resolución de problemas y el trabajo colaborativo, mediante el diseño de una herramienta de código abierto que reduzca la dependencia de software comercial.

En última instancia, el sistema no solo pretende ser una solución funcional para el cálculo matemático, sino una estrategia de innovación educativa que promueva un aprendizaje significativo e interdisciplinario, demostrando la potencia de Java para resolver desafíos del mundo real y contribuyendo a la democratización del acceso a recursos tecnológicos de alta calidad dentro del ámbito académico e institucional.

4. MARCO TEÓRICO

4.1. Fundamentos de programación y algoritmos

4.1.1. ¿Qué es la programación?

La programación es el arte de escribir código. El código se escribe para crear programas, y para escribirlo se requiere conocer sobre lenguajes de programación. Cada lenguaje es diferente entre sí, lo que quiere decir que admiten diferentes paradigmas. En palabras más simples, la programación es el proceso de crear un programa de computadora. (¿Qué es Programación? | TripleTen Ecuador s. f.)

4.1.2. ¿Qué es un algoritmo?

Es un conjunto de instrucciones definidas, ordenadas y acotadas para resolver un problema, realizar un cálculo o desarrollar una tarea. Es decir, un algoritmo es un procedimiento paso a paso para conseguir un fin. A partir de un estado e información iniciales, se siguen una serie de pasos ordenados para llegar a la solución de una situación. En programación, un algoritmo supone el paso previo a ponerse a escribir el código. Primero debemos encontrar la forma de obtener la solución al problema (definir el algoritmo informático), para luego, a través del código, poder indicarle a la máquina qué acciones queremos que lleve a cabo. De este modo, un programa informático no sería más que un conjunto de algoritmos ordenados y codificados en un lenguaje de programación para poder ser ejecutados en un ordenador. (Maluenda 2024)

4.1.3. ¿Qué es el pseudocódigo?

Es una herramienta utilizada para representar de manera abstracta y simplificada los pasos que un algoritmo debe seguir. Está diseñado para que cualquier persona, independientemente de su nivel de conocimiento técnico, pueda entender el proceso lógico detrás de la solución a un problema. Es comúnmente utilizado en la fase de diseño de software, ya que permite visualizar de forma clara el flujo del programa antes de implementarlo en un lenguaje de programación real.(Canelo, 2025)

Ejemplo de pseudocódigo:

```
Algoritmo PAR_IMPAR
    Escribir 'Por favor, teclea un número'
    Leer numero
    Si numero MOD 2=0 Entonces
        Escribir 'El número, ',numero,' es par'
    SiNo
        Escribir 'El número, ',numero,' es impar'
    FinSi
FinAlgoritmo
```

Figura 1 *Algoritmo de pseudocódigo*

Nota. El psudocódigo muestra la lógica de control en el operador MOD (Morales, 2019).

4.2.INTRODUCCIÓN A JAVA – COMPILACIÓN Y EJECUCIÓN

4.2.1. ¿Qué es Java?

Es un lenguaje de programación orientado a objetos y una plataforma de software ampliamente empleados que se ejecuta en miles de millones de computadoras portátiles, dispositivos móviles, consolas de juegos, dispositivos médicos, entre otros. Las reglas y la sintaxis de Java se basan en los lenguajes C y C++.

Una de las principales ventajas de desarrollar software con Java es su portabilidad. Una vez que haya escrito el código para un programa Java en una computadora portátil, podrá trasladarlo fácilmente a un dispositivo móvil (¿Que es Java? | IBM, 2021).

4.2.2. ¿Qué es la compilación y ejecución?

Se refieren al proceso esencial de traducir el código que escribes en un lenguaje de programación humano-legible a instrucciones que un ordenador puede entender y ejecutar. Este proceso es fundamental para que tus ideas se conviertan en programas funcionales.

El proceso de compilación

El proceso de compilación implica transformar el código fuente escrito en un lenguaje de programación en código ejecutable por la máquina. En muchos lenguajes, como C++, el código fuente se traduce a un lenguaje de máquina específico para el sistema operativo. Esto crea un archivo ejecutable que puede ser invocado y ejecutado directamente por el sistema operativo.

Ejecución de programas

Una vez que el código ha sido compilado en un programa ejecutable, el siguiente paso es ejecutar el programa. Durante la ejecución, el sistema operativo carga el programa en la memoria y sigue las instrucciones del programa. Esto lleva a la realización de tareas específicas y produce resultados basados en el código que has escrito (Casero, 2023).

4.3.INSTALACIÓN DEL JDK, IDE Y HOLA MUNDO

4.3.1. ¿Qué es el JDK?

El Kit de Desarrollo de Java (JDK) es básicamente un entorno de desarrollo de software proporcionado por Oracle y otros proveedores responsables de la creación de aplicaciones Java . Consiste en un paquete de herramientas necesarias para desarrollar, depurar y ejecutar programas

en Java. Para escribir una aplicación Java, el JDK es el primer componente que se instala. El JDK consta de API y herramientas importantes que se requieren para crear, empaquetar y documentar aplicaciones (Alam, 2025).

4.3.2. ¿Qué es un IDE?

Un entorno de desarrollo integrado (IDE) es una aplicación de software que ayuda a los programadores a desarrollar código de software de manera eficiente. Aumenta la productividad de los desarrolladores al combinar capacidades como editar, crear, probar y empaquetar software en una aplicación fácil de usar. Así como los escritores utilizan editores de texto y los contables, hojas de cálculo, los desarrolladores de software utilizan IDE para facilitar su trabajo (¿Qué es un IDE? - Explicación de los entornos de desarrollo integrado - AWS s. f.).

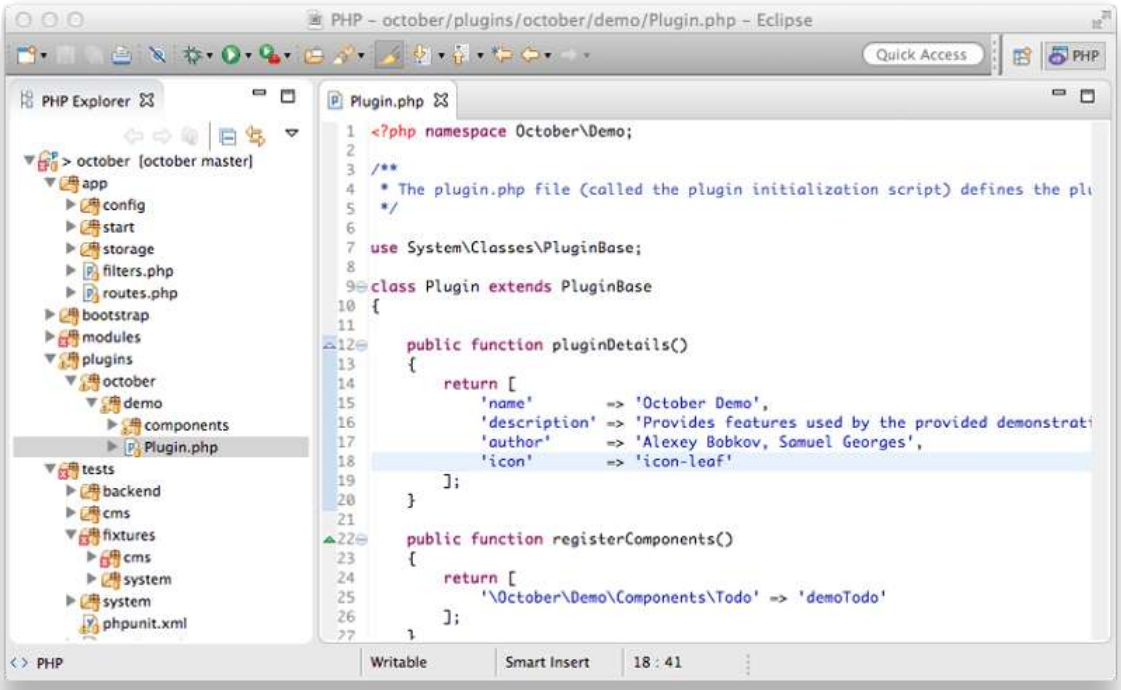


Figura 2 *IDE Eclipse*

Nota. Ofrece un entorno de desarrollo integrado (Bobkov, 2014).

4.4.Operadores y estructuras condicionales

4.4.1. ¿Qué son los operadores aritméticos?

Los operadores aritméticos en programación son símbolos que representan las operaciones matemáticas básicas como la suma, resta, multiplicación y división. Estos operadores te permiten realizar cálculos entre variables y valores, facilitando la manipulación y el análisis de datos.

Además de estas operaciones básicas, los operadores aritméticos también juegan un papel crucial en la formación de expresiones más complejas, permitiendo combinar múltiples operaciones en una sola línea de código. Esto no solo mejora la legibilidad del código, sino que también optimiza el rendimiento al minimizar la cantidad de pasos necesarios para obtener un resultado. Al entender y utilizar eficazmente estos operadores, los programadores pueden desarrollar algoritmos más eficientes y precisos, esenciales para tareas desde cálculos financieros hasta el procesamiento de datos científicos (Maldonado, 2024).

```
1   Operator
2   +          Addition
3   -          Subtraction
4   *          Multiplication
5   **         Exponentiation
6   /          Division
7   %          Modulus
8   ++         Increment
9   --         Decrement
```

Figura 3 *Operadores Aritméticos*

Nota. Tipos de Operadores Aritméticos en Programación (Bernal, 2020).

4.4.2. ¿Qué es la estructura if–else?

La estructura if-else se utiliza en Java para tomar decisiones sobre un valor preexistente. Es decir, si una condición basada en ese valor preexistente se cumple (si esa condición es verdadera), el programa tomará un camino. Si no (si esa condición es falsa), el programa tomará otro. A esa condición se la llama condición booleana, ya que un valor booleano (*boolean*) es aquel que solo puede tomar como valor *true* (verdadero) o *false* (falso).

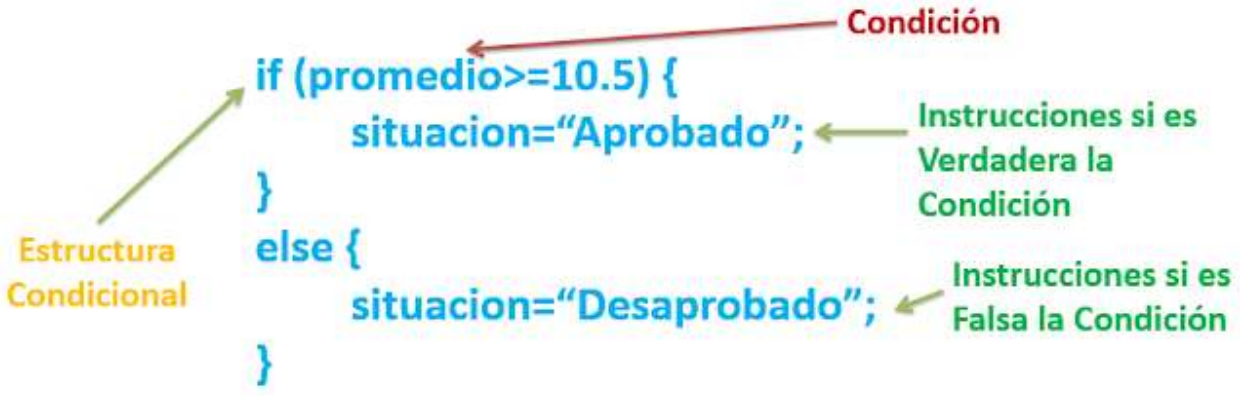


Figura 4 *if – else*

Nota. Declaración de un if – else (Arcila, 2015).

4.4.3. ¿Qué es el operador ternario?

El operador ternario es una alternativa al condicional if/else de una forma mucho más compacta y breve, que en muchos casos resulta más legible. Sin embargo, hay que tener cuidado, porque su sobreutilización puede ser contraproducente y producir un código más difícil de leer.

```
let nota = 7;
console.log("He realizado mi examen. Mi resultado es el siguiente:");

// Operador ternario: (condición ? verdadero : falso)
let calificacion = nota < 5 ? "suspendido" : "aprobado";

console.log("Estoy", calificacion);
```

Figura 5 *Operador Ternario*

Nota. Esta es una alternativa más compacta y breve (Pérez, 2022).

4.5. Modularización

Concepto

Es el concepto de diseñar soluciones a partir de módulos independientes e intercambiables que pueden combinarse, reemplazarse o ampliarse según sea necesario.

Definición

Es el proceso de dividir un sistema o programa complejo en partes más pequeñas, independientes e intercambiables, llamadas módulos.

4.5.1. Métodos de uso

Se implementa mediante técnicas de diseño de software que dividen el sistema en componentes manejables utilizando métodos como procedimientos, funciones y la programación orientada a objetos.

Abstracción Funcional: Consiste en aislar bloques de código que realizan una tarea específica.

Programación Orientada a Objetos: Es un método de diseño avanzado que utiliza conceptos de modularidad de manera intrínseca.

Módulos y Paquetes: Los lenguajes de programación modernos (como Python, JavaScript, Java) ofrecen construcciones específicas para la modularización a gran escala.

Arquitectura de Microservicios: En sistemas distribuidos y a gran escala, la modularización se extiende a nivel de arquitectura.

Figura 6 *Modularización*

Nota. Estructura de la Modularización (Uriarte, 2021).

4.5.2. Modularización en Variable

En el ámbito de una variable se refiere a la región del programa donde esa variable puede ser utilizada, accedida o modificada define la visibilidad y el ciclo de vida de los identificadores dentro del código. Es fundamental en la modularización, ya que controla qué módulos pueden "ver" o interactuar con los datos definidos en otros módulos.

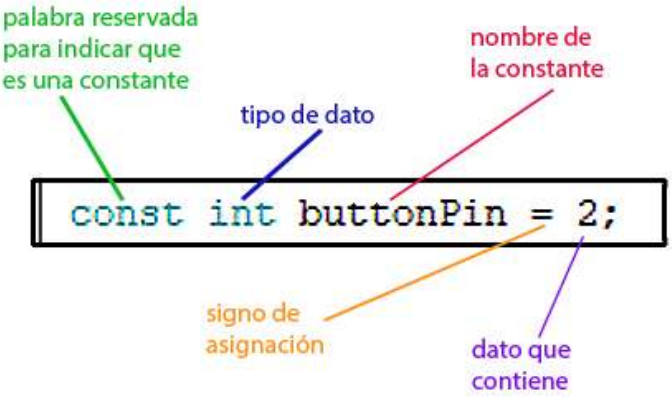


Figura 7 *Modularización en una Variable*

Nota. Declaración de una Variable (Tecno-Comunicación, 2016).

4.6.Sobrecarga de Métodos (scope)

Es una característica del polimorfismo que permite definir múltiples métodos dentro de la misma clase que comparten el mismo nombre pero que se diferencian por sus firmas.

¿Como Funciona?

Para que la sobrecarga sea válida cada método con el mismo nombre debe tener una firma de método única.

El nombre del método.

El número de parámetros.

El tipo de datos de los parámetros.

El orden de los tipos de parámetros.

Recursividad Básica

La recursividad básica es un concepto fundamental en informática y matemáticas donde una función o procedimiento se define en términos de sí misma, esto significa que la función se llama a sí misma repetidamente para resolver un problema.

4.6.1. Componentes clave

Caso Base: Cuando se alcanza el caso base, la función devuelve un resultado sin realizar otra llamada recursiva.

Paso Recursivo: Es la parte donde la función se llama a sí misma, pero siempre con un conjunto de datos o un problema que es ligeramente más pequeño o más simple que el original.

4.7.Arreglos Unidimensionales

Comúnmente conocidos como vectores o listas son estructuras de datos que almacenan una colección de elementos del mismo tipo de datos bajo un único nombre de variable.

Características

Homogeneidad: Todos los elementos almacenados deben ser del mismo tipo.

Tamaño Fijo: El tamaño del arreglo se define al crearlo y no puede cambiar durante la ejecución del programa.

Acceso Directo: Se puede acceder a cualquier elemento del arreglo de manera inmediata si se conoce su posición.

Indexación: Los elementos se identifican por su posición.

```
1  class Bidimensional{
2      public static void main(String[] args){
3          int a[][] = {{4,7},{3,4}};
4          mostrar(a);
5      }
6
7      static void mostrar(int[][] arreglo){
8          for(int fila=0; fila<arreglo.length; fila++){
9              for(int columna=0; columna<arreglo.length; columna++){
10
11             }
12         }
13     }
14 }
```

Figura 8 *Arreglos Unidimensionales*

Nota. Taller de Estructuras en Java - 05 Arreglos Multidimensionales (Lozada 2013).

4.7.1. Declaración

Es un término en programación que se refiere al acto de presentar un identificador como el nombre de una variable, función, clase o módulo al compilador o intérprete, estableciendo su existencia y sus características esenciales, como su tipo de dato.

Tipos de declaración

Declaración de Variables.

Declaración de Funciones o Métodos.

Declaración de Clases o Estructuras

Inicialización y Recorrido

Es el proceso de asignar un valor inicial a una variable o a los elementos de una estructura de datos en el momento de su declaración o creación.

La inicialización es el proceso de asignar un valor inicial a una variable o a los elementos de una estructura de datos en el momento de su declaración o creación. El propósito principal es asegurar que la estructura de datos no contenga datos residuales innecesarios y esté lista para ser utilizada de manera predecible.

Métodos

En la declaración

Manual o por Recorrido

El recorrido es el proceso de visitar, examinar o procesar sistemáticamente cada uno de los elementos de una estructura de datos exactamente una vez.

Al recorrer un arreglo unidimensional, se suele usar un bucle y un índice para acceder a cada posición secuencialmente, desde el primer elemento hasta el último.

4.8.Recorrido y Búsqueda de arreglos

El recorrido y la búsqueda son dos operaciones que se realizan sobre arreglos para acceder y analizar los datos almacenados. Ambas operaciones suelen implicar el uso de bucles.

4.8.1. Recorridos de Arreglos

El recorrido es la acción de visitar cada elemento de un arreglo de principio a fin para realizar alguna operación con su valor.

Como, por ejemplo:

Imprimir todos los elementos.

Sumar todos los valores.

Modificar cada elemento (ej: multiplicar todos los salarios por 1.10).

Encontrar el valor máximo o mínimo.

Métodos

4.9.Método de Recorrido: Bucle for

El bucle for tradicional es el método más común utilizando una variable de índice (i) que comienza en 0 y se incrementa hasta justo antes de la longitud total del arreglo.

4.9.1. Bucle for-each

Muchos lenguajes modernos ofrecen una sintaxis más limpia para cuando solo necesitas leer los valores sin usar el índice directamente.

```
for (int i = 0; i < 5; i++) {
    System.out.println("Iteración número: " + i);
}
```

Figura 9 *Bucle for – each*

Nota. El código ilustra la sintaxis básica para repetir una acción un número determinado de veces (HACK A BOSS, 2025)

4.10. Búsqueda de Arreglos

La búsqueda es el proceso de encontrar la ubicación de un elemento específico dentro del arreglo, o simplemente verificar si dicho elemento existe.

Existen dos algoritmos principales para la búsqueda

4.10.1. Búsqueda Lineal

Este es el método más simple y funciona recorriendo el arreglo elemento por elemento desde el principio hasta el final, comparando cada valor con el elemento que se busca.

4.10.2. Búsqueda Binaria

La búsqueda binaria es un algoritmo mucho más eficiente, pero requiere que el arreglo esté previamente ordenado.

Revisa el elemento central del arreglo; si el valor buscado es menor, busca en la mitad izquierda; si es mayor, busca en la mitad derecha, y repite el proceso hasta encontrarlo.

4.11. Ordenamiento Básico (Burbuja)

El método funciona comparando pares de elementos adyacentes repetidamente y, si están en el orden incorrecto, por ejemplo, el primero es mayor que el segundo si se busca ordenar de forma ascendente, los intercambia de posición.

Este proceso se repite hasta que no se necesiten más intercambios, lo que indica que el arreglo está completamente ordenado.

4.11.1. Cómo Funciona

Primera pasada (recorrido):

Compara (5, 1): Están en orden incorrecto, intercambia -> [1, 5, 4, 2, 8]

Compara (5, 4): Están en orden incorrecto, intercambia -> [1, 4, 5, 2, 8]

Compara (5, 2): Están en orden incorrecto, intercambia -> [1, 4, 2, 5, 8]

Compara (5, 8): Están en orden correcto, no hace nada -> [1, 4, 2, 5, 8]

Resultado de la primera pasada: El número más grande (8) está ahora fijo en su posición final.

Segunda pasada:

Compara (1, 4): Correcto -> [1, 4, 2, 5, 8]

Compara (4, 2): Incorrecto, intercambia -> [1, 2, 4, 5, 8]

Compara (4, 5): Correcto -> [1, 2, 4, 5, 8]

Resultado de la segunda pasada: El segundo número más grande (5) está fijo.

Tercera pasada:

Compara (1, 2): Correcto -> [1, 2, 4, 5, 8]

Compara (2, 4): Correcto -> [1, 2, 4, 5, 8]

Resultado de la tercera pasada: El arreglo está ordenado.

4.12. Arreglos Multidimensionales

Son estructuras de datos que extienden el concepto de los arreglos unidimensionales que permiten almacenar datos de manera estructurada en más de una dimensión generalmente como una cuadrícula, tabla o matriz.

4.13. Arreglo Bidimensional (Matriz)

Una matriz es esencialmente un "arreglo de arreglos". Para acceder a un elemento específico, se requieren dos índices: uno para la fila y otro para la columna.

Los arreglos multidimensionales son ideales para modelar:

Tableros de juego: Ajedrez, Buscaminas, Tres en raya.

Imágenes: Cada celda (píxel) almacena un valor de color (RGB).

Coordenadas geográficas: Almacenar datos basados en latitud y longitud.

Sistemas de notas: Filas para estudiantes, columnas para materias/exámenes.

4.14. Matrices y su Manipulación

Son la implementación más común de los arreglos bidimensionales y son fundamentales para organizar datos en forma de cuadrícula, la manipulación de matrices implica realizar operaciones específicas sobre sus elementos, que van desde simples lecturas hasta cálculos matemáticos complejos.

4.15. Operaciones comunes de manipulación de matrices

Suma de Matrices: Para sumar dos matrices deben tener exactamente las mismas dimensiones, se recorren ambas matrices simultáneamente y se suma elemento por elemento, almacenando el resultado en una tercera matriz.

4.15.1. Multiplicación de Matrices

La multiplicación de matrices es más compleja que la suma. No se multiplican elemento por elemento. Para multiplicar la Matriz A por la Matriz B, el número de columnas de A debe ser igual al número de filas de B.

4.15.2. Trasposición de Matriz

La transposición implica intercambiar las filas por las columnas.

Búsqueda de Valores: Similar a los arreglos unidimensionales, se pueden buscar elementos específicos. En matrices se busca un valor máximo, mínimo o un valor específico, reportando su ubicación.

Manipulación de Arreglos "Jagged": En muchos lenguajes, los arreglos bidimensionales no tienen que ser perfectamente rectangulares. Se pueden crear arreglos donde cada fila tiene una longitud diferente.

$$A = \begin{pmatrix} 2 & 0 & 1 \\ 3 & 0 & 0 \\ 5 & 1 & 1 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$A+B = \begin{pmatrix} 2+1 & 0+0 & 1+1 \\ 3+1 & 0+2 & 0+1 \\ 5+1 & 1+1 & 1+0 \end{pmatrix} = \begin{pmatrix} 3 & 0 & 2 \\ 4 & 2 & 1 \\ 6 & 2 & 1 \end{pmatrix}$$

$$A-B = \begin{pmatrix} 2-1 & 0-0 & 1-1 \\ 3-1 & 0-2 & 0-1 \\ 5-1 & 1-1 & 1-0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & -2 & -1 \\ 4 & 0 & 1 \end{pmatrix}$$

Figura 10 *Trasposición de matriz*

Nota. La imagen ilustra el procedimiento paso a paso para operar con matrices (Evilnapsis, 2019).

4.16. Cadenas de caracteres (String)

Son un tipo de dato utilizado en programación para representar y almacenar secuencias de texto, como palabras, frases, nombres de usuario o cualquier combinación de letras, números y símbolos.

```java
public class StringtoChar {

    public static void main (String[] args){
        String cadena = "Suscribete a Código Root";
        char [] caracteres = cadena.toCharArray();

        for (int i = 0; i < caracteres.length; i++) {
            System.out.println( "Posición[" + i + "] "  + caracteres [i]);
        }

    }

}
```

Figura 11 *Cadena de Caracteres String*

Nota. muestra un ejemplo de programación en **Java** utilizado para enseñar cómo convertir una cadena de texto en un arreglo de caracteres (Código Root, 2022).

4.16.1. Características Clave de las Cadenas de Caracteres

Secuencia de Caracteres: Una cadena es una colección ordenada de caracteres individuales.

Inmutabilidad: En lenguajes como Java, una vez que se crea una cadena, no se puede modificar su contenido original y cualquier operación que parezca modificarla en realidad crea una nueva cadena con los cambios aplicados.

Representación: Internamente, los caracteres se almacenan utilizando estándares de codificación como ASCII o, más comúnmente hoy en día, Unicode para manejar texto de todos los idiomas.

Acceso mediante Índice: Al igual que los arreglos, los caracteres dentro de una cadena pueden ser accedidos individualmente usando su posición (índice, que comienza en 0).

Métodos Principales de String

Consulta de Información: Permiten obtener datos sobre la cadena sin modificarla.

Métodos de Búsqueda y Localización: Ayudan a encontrar subcadenas o caracteres específicos dentro de una cadena más grande.

Método de Extracción y Transformación: Crean nuevas cadenas basadas en la original.

Métodos de División y Unión: Permiten interactuar con arreglos y la cadena.

Factorización y productos notables

5. MATEMÁTICA

5.1.Trinomio cuadrado perfecto

Un trinomio cuadrado perfecto es una expresión algebraica de tres términos que resulta de elevar un binomio al cuadrado. Cumple con dos características: los dos primeros y los dos últimos términos son cuadrados perfectos, y el término del medio es el doble producto de las raíces cuadradas de esos dos términos.

$$9 + 6x + x^2 = (3 + x)^2$$

$$\downarrow \quad \uparrow \quad \downarrow$$

$$3^2 \quad 2 \cdot 3 \cdot x \quad x^2$$

Figura 12 *Trinomio Cuadrado Perfecto*

Nota. Ilustra el proceso de factorización de un trinomio cuadrado perfecto (Spanish GED 365, 2014).

5.1.1. Trinomio de la forma ax2 + bx +c

Este tipo de trinomio se diferencia del anterior debido a que el termino al cuadrado (x^2) se encuentra precedido por un coeficiente diferente de uno (debe ser positivo).

$$x^2 + 5x + 6 = (x+3)(x+2)$$
$$x^2 - 2x - 24 = (x-6)(x+4)$$
$$x^2 - 8x + 15 = (x-5)(x-3)$$
$$-x^2 + 6x - 8 = -(x^2 - 6x + 8)$$
$$= -(x-4)(x-2)$$
$$= (-x+4)(x-2)$$

Figura 13 *Trinomio de la forma ax2 + bx + c*

Nota. La imagen nos muestra ejemplos resueltos de factorización de trinomios (Carlos, 2022).

5.1.2. Suma de cubos

La suma de cubos perfectos se puede factorizar así:

$$a^3 + b^3 = (a + b)(a^2 - ab + b^2)$$

Es una multiplicación de polinomios resultado de un binomio, suma de las bases, multiplicado por un trinomio, resultante del cuadrado de la primera raíz, menos el producto de las dos raíces, más el cuadrado de la segunda raíz.

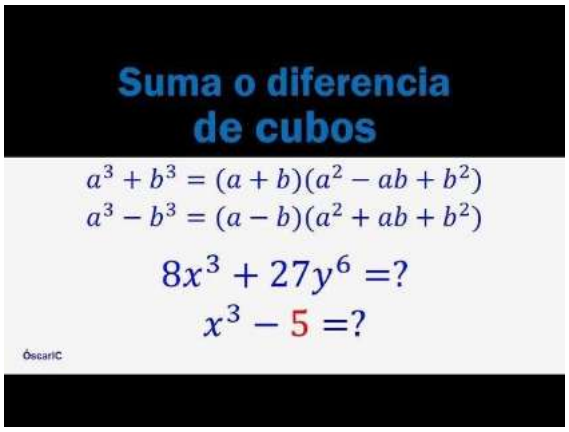


Figura 14 *Suma de Cubos*

Nota. Presenta las fórmulas y ejemplos de la suma o diferencia de cubos (ÓscarC, 2014).

5.2.Diferencia de cubos

La fórmula de la diferencia de cubos en álgebra se utiliza para calcular el valor de la expresión algebraica (a^3 - b^3). En pocas palabras, se aplica para igualar la diferencia de dos valores cúbicos.

Por lo tanto, la fórmula para la diferencia de cubos en álgebra se expresa como: $a^3 - b^3 = (a - b)(a^2 + ab + b^2)$

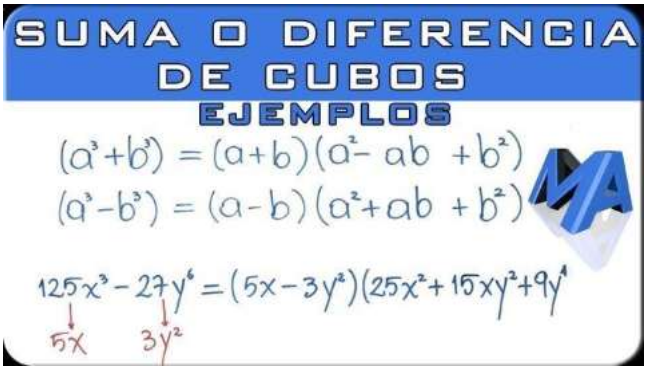


Figura 15 Diferencia de Cubos

Nota. Esta imagen fue creada por el canal educativo (Lozada, 2016).

5.3.Ecuaciones de primer grado

5.3.1. Lenguaje algebraico

El lenguaje algebraico es un sistema que usa números, letras y símbolos matemáticos para expresar ideas y problemas matemáticos de forma general. Las letras representan valores desconocidos o variables, mientras que los números representan constantes. Permite transformar enunciados del lenguaje común en expresiones concisas

5.3.2. Definición ecuación lineal simple

Una ecuación lineal es una ecuación en la que la mayor potencia de la variable siempre es 1. También se conoce como ecuación de un grado. La forma estándar de una ecuación lineal con una variable es $Ax + B = 0$. En ella, x es una variable, A es un coeficiente y B es una constante.

La forma estándar de una ecuación lineal con dos variables es Ax + By = C. En ella, x e y son variables, A y B son coeficientes y C es una constante.

Reglas básicas de despeje de una variable

Reglas para despejar:

1.- Lo que está sumando pasa restando.

2.- Lo que está restando pasa sumando

3.- Lo que está multiplicando pasa dividiendo

4.- Lo que está dividiendo pasa multiplicando

5.- Si está con exponente pasa con raíz.

5.4.Método de Resolución de Sistema de Ecuaciones Lineales

5.4.1. Método por determinantes

El método de determinantes, también conocido como regla de Cramer, se utiliza para resolver sistemas de ecuaciones lineales. El método implica calcular tres determinantes: el del sistema (que utiliza los coeficientes de las incógnitas), el de la variable (reemplazando los coeficientes de la incógnita por los términos independientes) y el de la otra variable. Luego, se divide el determinante de cada variable entre el determinante del sistema para encontrar el valor de cada incógnita.

$$\Delta S = \begin{vmatrix} 5 & -2 \\ -3 & 7 \end{vmatrix} = 5 * 7 - (-3 * -2) = \boxed{29}$$

$$\Delta X = \begin{vmatrix} -2 & -2 \\ -22 & 7 \end{vmatrix} = -2 * 7 - (-22 * -2) = \boxed{-58}$$

$$\Delta Y = \begin{vmatrix} 5 & -2 \\ -3 & -22 \end{vmatrix} = 5 * (-22) - (-3 * -2) = \boxed{-116}$$

Figura 16 Determinantes

Nota. Esta imagen muestra la aplicación de la regla de Cramer para resolver un sistema de ecuaciones de 2 x 2 (Lozada 2022).

5.4.2. Método grafico

El método gráfico para resolver un sistema de ecuaciones lineales consiste en representar ambas ecuaciones como rectas en un plano cartesiano. La solución del sistema es el punto donde estas dos rectas se cruzan son la solución común para ambas ecuaciones. Si las rectas son paralelas, no hay solución; si son la misma recta, hay infinitas soluciones.

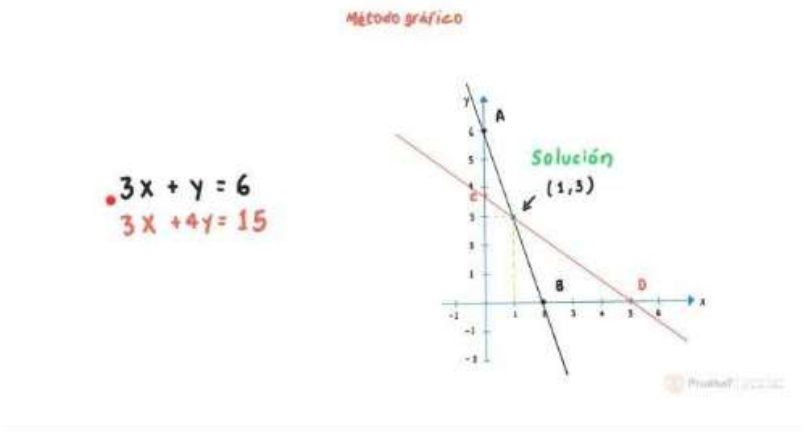


Figura 17 Método Gráfico

Nota. Muestra la resolución de un sistema de ecuaciones lineales de 2x2 (PruébaT, 2016).

5.5.Ecuaciones de 2do grado

Ecuación cuadrática o ecuación de 2do grado es la que tiene una incógnita es aquella que tiene la expresión general

ax2+bx+c=0,a≠0$\boldsymbol{ax2+bx+c=0, a\neq 0}$

Las ecuaciones de segundo grado, también llamadas ecuaciones cuadráticas, son aquellas en el que el mayor grado es 2 es decir, el mayor exponente de la ecuación es la variable x elevada al cuadrado, la ecuación se denomina en diferentes terminos el término cuadrático, ax2, que se compone de un coeficiente a y la variable x elevada al cuadrado el término lineal bx que se compone del coeficiente b y la variable x y el término independiente c.

Resolución de ecuación segundo grado mediante la formula general.

Para resolver ejercicios propuestos, se utilizara la fórmula general para ecuaciones de segundo grado

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Ejercicios resueltos con formula general

$$x^2 + 6 = 0$$

Identificamos los valores de a b y c

a=1, b=0, c=6

Sustituimos valores en la formula general y resolvemos

$$x=\frac{-0\pm\sqrt{(0)^2-(4)(1)(6)}}{(2)(1)}$$

$$x=\frac{\pm\sqrt{-24}}{2}$$

$$\frac{\pm\sqrt{-24}}{2}\notin\mathbb{R}$$

5.6.Resolución por Factorización.

Este es un producto de dos expresiones, y es igual a cero. Observa que cualquier valor de [x] que haga que [(x-1)] o [(x+3)] sea cero, hará que el producto sea cero. (x-1)&(x+3)=0

El sustituir [x=1] o bien [x=-3] en la ecuación tiene por resultado la ecuación verdadera [0=0] así que ambos valores son soluciones de la ecuación.

Función y sus características

Dominio de una función

El dominio de una función está formado por todos los elementos que tienen imagen es decir son los valores de x que podemos sustituir en la regla de correspondencia de una función para obtener el valor correspondiente de f(x)

$$D=\{x\,\epsilon\,\mathbb{R}\,/\,\exists\,f(x)\}$$

Que significa que el dominio de una función son aquellos valores de x que pertenecen a los números reales para los cuales existe un valor asociado de la función f(x)

5.7.Rango de una función

Se denomina rango o recorrido de una función al conjunto de los valores reales que toma la variable y o f

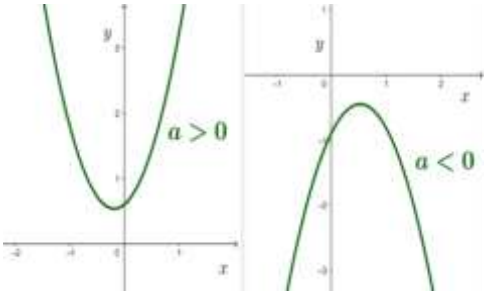


Figura 18 Función cuadrática

Nota. Muestra dos gráficas de funciones cuadráticas (EDEA, 2014).

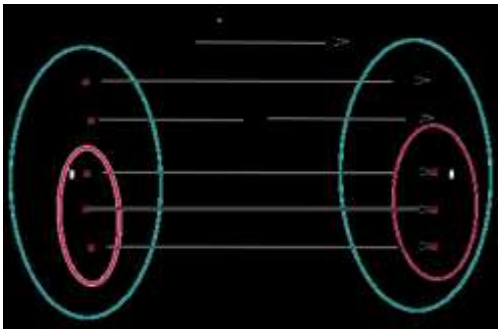


Figura 19 Diagrama Sagital

Nota. La imagen nos explica el concepto de **rango o recorrido de una función** (Superprof, 2014).

5.8.Intervalos de positividad y negatividad.

Una función f es positiva en un intervalo si ocurre que f(x)=0 para toda x es negativa en si f(x)=0 para toda x la parte positiva de una función se ubica por encima del eje x y la negativa por debajo.

(Effenberger, 2016)

5.9.Monotonía de la Función

La monotonía de una función consiste en ver en los puntos del dominio donde esta función crece o decrece.

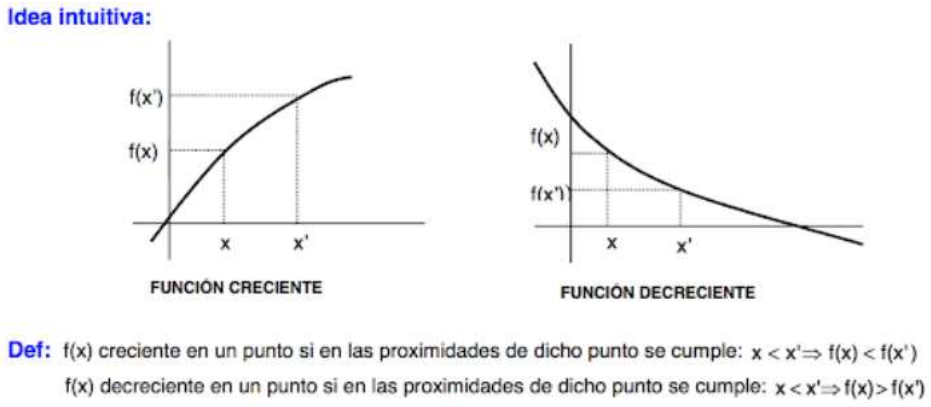


Figura 20 Funciones Crecientes y Decrecientes

Nota. Nos presenta una idea intuitiva y la definición formal de funciones crecientes y decrecientes (Gonzáles, 2014).

5.10. Asíntota verticales y horizontales

5.10.1. Asíntotas Verticales

Para encontrar las asíntotas verticales igualamos el denominador a cero y encontramos las soluciones o ceros

Para encontrar las asíntotas horizontales necesitamos comparar el grado del numerador(GN) y con el grado del denominador (GD) donde CPN es el coeficiente principal del numerador y CPD es el coeficiente principal del denominador.

5.11. Función Cuadrática

5.11.1. Definición y Representación

Las funciones polinómicas son aquellas constituidas por un polinomio un ejemplo de estas es la función cuadrática o de segundo grado representada con una gráfica de parábola y la siguiente ecuación

$$f(x)=ax^2 + bx + c$$

5.12. Concavidad de una Función Cuadrática

La concavidad de una función cuadrática se determina por el signo del coeficiente (a) de la ecuación (f(x)=ax2+bx+c) si (a=0) la parábola es cóncava hacia arriba (tiene forma de U y un valor mínimo en el vértice)

5.13. Vértice de la Función Cuadrática

Por el vértice pasa el eje de simetría de la parábola es decir, cuando el coeficiente del término es positivo el vértice será el punto más bajo de la gráfica

Eje de simetría

Es una línea recta imaginaria que divide una figura en dos partes que son imágenes especulares una de la otra de modo que si doblaras la figura por esa línea ambas mitades coincidirían perfectamente.

5.14. Raíces reales de una Función

Las raíces reales se refieren a los valores numéricos que satisfacen una ecuación en forma de f(x) = 0 donde f(x) es una función real una raíz real es un valor de x para el cual la función se anula y toma el valor de cero en otras palabras es el punto en el que la gráfica de la función cruza el eje x.

6. RESULTADOS

Primero empezamos por declarar cada variable y método que usamos en nuestra calculadora, junto con formulas que se iban a calcular, como lo serian las ecuaciones de Primer grado, segundo grado, el Área del Cuadrado, Triangulo, Rectángulo y el Radio del Círculo, cada uno separado y con su propia ventana.

PAIT G3 - pait_grupo/src/paitgrupo3/interfaz.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

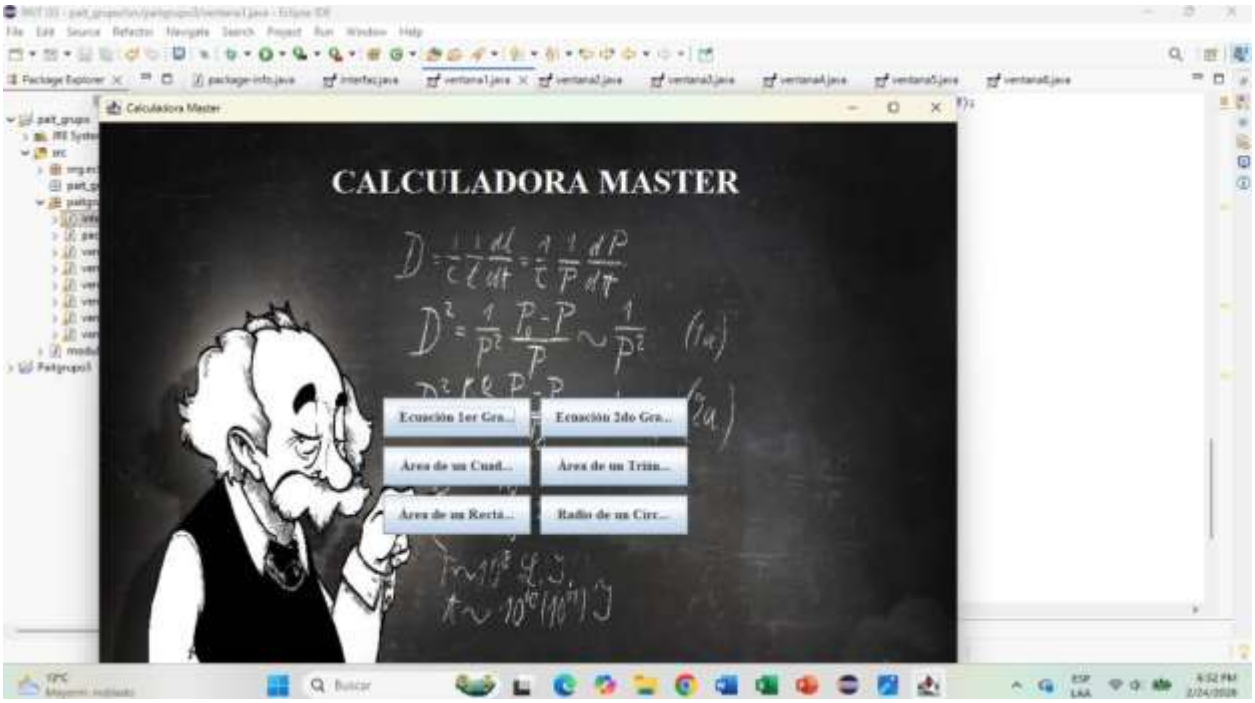
Package Explorer | package-info.java | interfaz.java | ventana1.java | ventana2.java | ventana3.java | ventana4.java | ventana

- pait_grupo
  - JRE System Library [JavaSE-
  - src
    - org.eclipse.wb.swing
    - pait_grupo
    - paitgrupo3
      - interfaz.java
      - package-info.java
      - ventana1.java
      - ventana2.java
      - ventana3.java
      - ventana4.java
      - ventana5.java
      - ventana6.java
    - module-info.java
- Paitgrupo3

```java
package paitgrupo3;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;

public class interfaz extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JLabel lblTitulo;
    private JPanel panelBotones;

    private JButton btnEcuacion1, btnEcuacion2, btnCuadrado, btnTriangulo, btnRectangulo, btnCirculo;

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    interfaz frame = new interfaz();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    public interfaz() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 950, 750);
        setTitle("Calculadora Master");

        contentPane = new PanelConFondo("C:\\Users\\artur\\Downloads\\fondo1.1.jpg");
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
```

Source | Design

Writable | Smart Insert | 151 : 2 : 5499



PAIT G3 - pait_grupo/src/paitgrupo3/ventana1.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

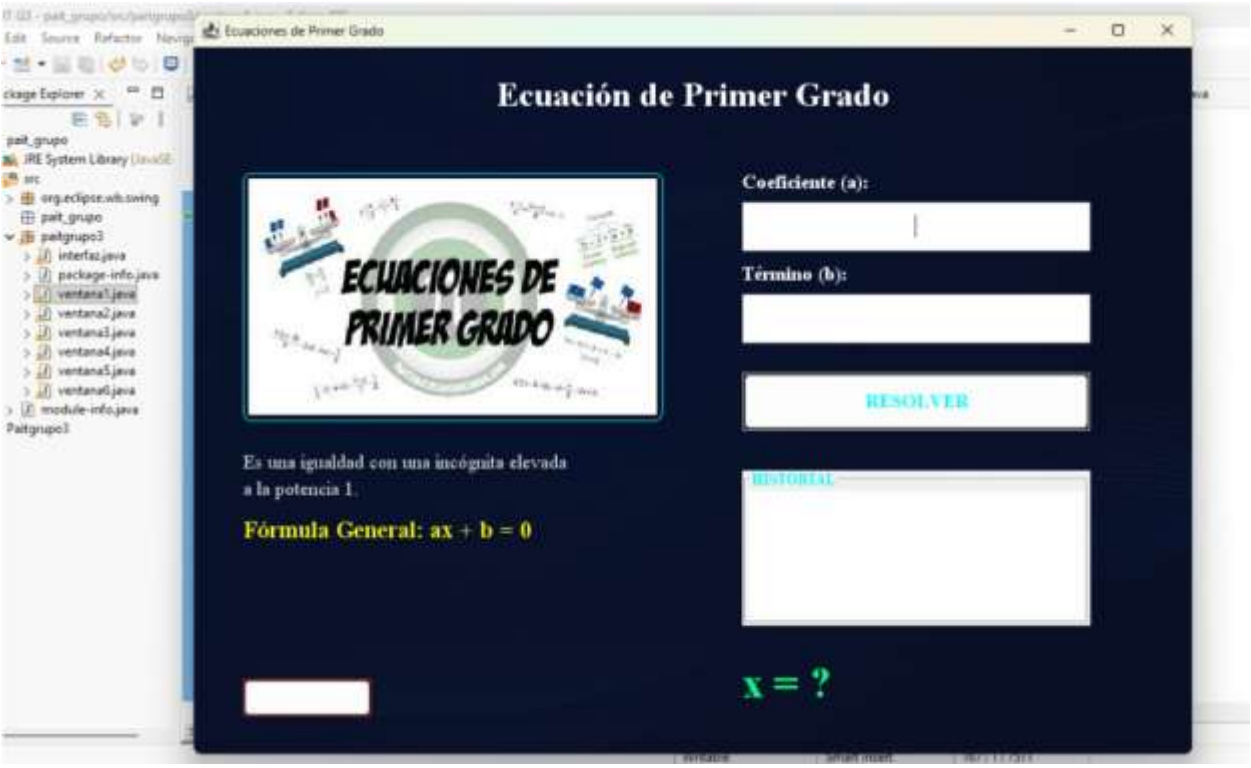
Package Explorer

- pait_grupo
  - JRE System Library [JavaSE-
  - src
    - org.eclipse.wb.swing
    - pait_grupo
    - paitgrupo3
      - interfaz.java
      - package-info.java
      - ventana1.java
      - ventana2.java
      - ventana3.java
      - ventana4.java
      - ventana5.java
      - ventana6.java
    - module-info.java
- Paitgrupo3

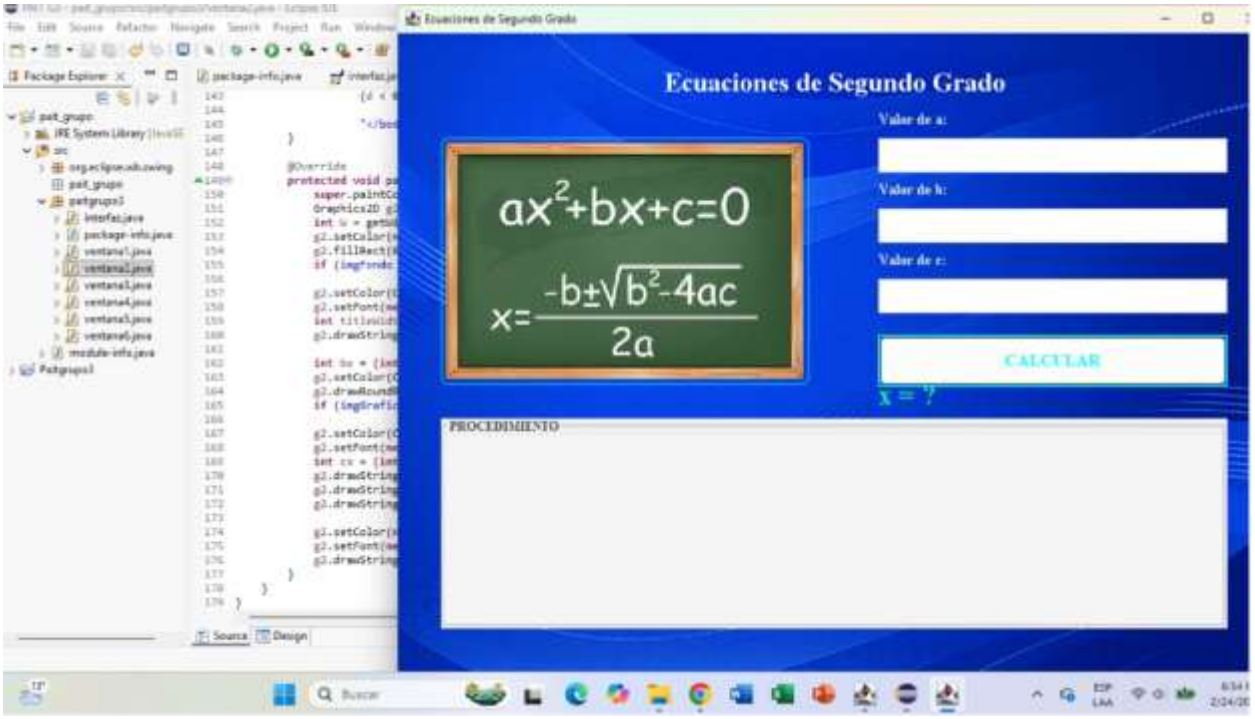
package-info.java | interfaz.java | ventana1.java | ventana2.java | ventana3.java | ventana4.java

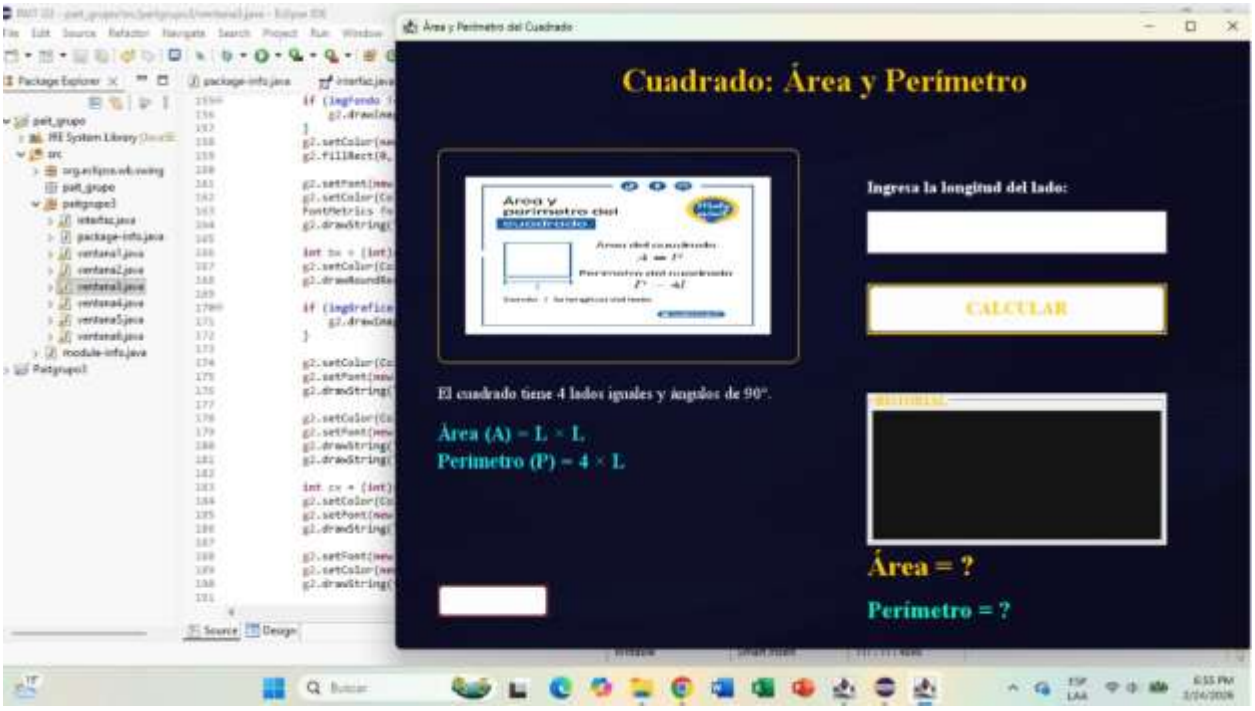
```
122                 JOptionPane.showMessageDialog(this, "Ingresa números válidos.", "Error", JOptionPane
123             }
124             repaint();
125         }
126
127         @Override
128         protected void paintComponent(Graphics g) {
129             super.paintComponent(g);
130             Graphics2D g2 = (Graphics2D) g;
131             g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
132
133             int W = getWidth(), H = getHeight();
134             g2.drawImage(imgFondo, 0, 0, W, H, this);
135             g2.setColor(new Color(10, 15, 30, 235));
136             g2.fillRect(0, 0, W, H);
137
138             g2.setColor(Color.WHITE);
139             g2.setFont(new Font(FUENTE_PRINCIPAL, Font.BOLD, 32));
140             int titleWidth = g2.getFontMetrics().stringWidth("Ecuación de Primer Grado");
141             g2.drawString("Ecuación de Primer Grado", (W - titleWidth) / 2, H / 12);
142
143             int bx = (int)(W*0.05), by = (int)(H*0.18), bw = (int)(W*0.42), bh = (int)(H*0.35);
144             g2.setColor(Color.CYAN);
145             g2.drawRoundRect(bx, by, bw, bh, 15, 15);
146             g2.drawImage(imgGrafica, bx+5, by+5, bw-10, bh-10, this);
147
148             g2.setColor(new Color(210, 210, 210));
149             g2.setFont(new Font(FUENTE_PRINCIPAL, Font.PLAIN, 18));
150             g2.drawString("Es una igualdad con una incógnita elevada", bx, by + bh + 45);
151             g2.drawString("a la potencia 1.", bx, by + bh + 70);
152
153             g2.setColor(Color.YELLOW);
154             g2.setFont(new Font(FUENTE_PRINCIPAL, Font.BOLD, 22));
155             g2.drawString("Fórmula General: ax + b = 0", bx, by + bh + 110);
156
157             int cx = (int)(W*0.55), iw = (int)(W*0.35), ih = (int)(H*0.07);
158             g2.setColor(Color.WHITE);
```

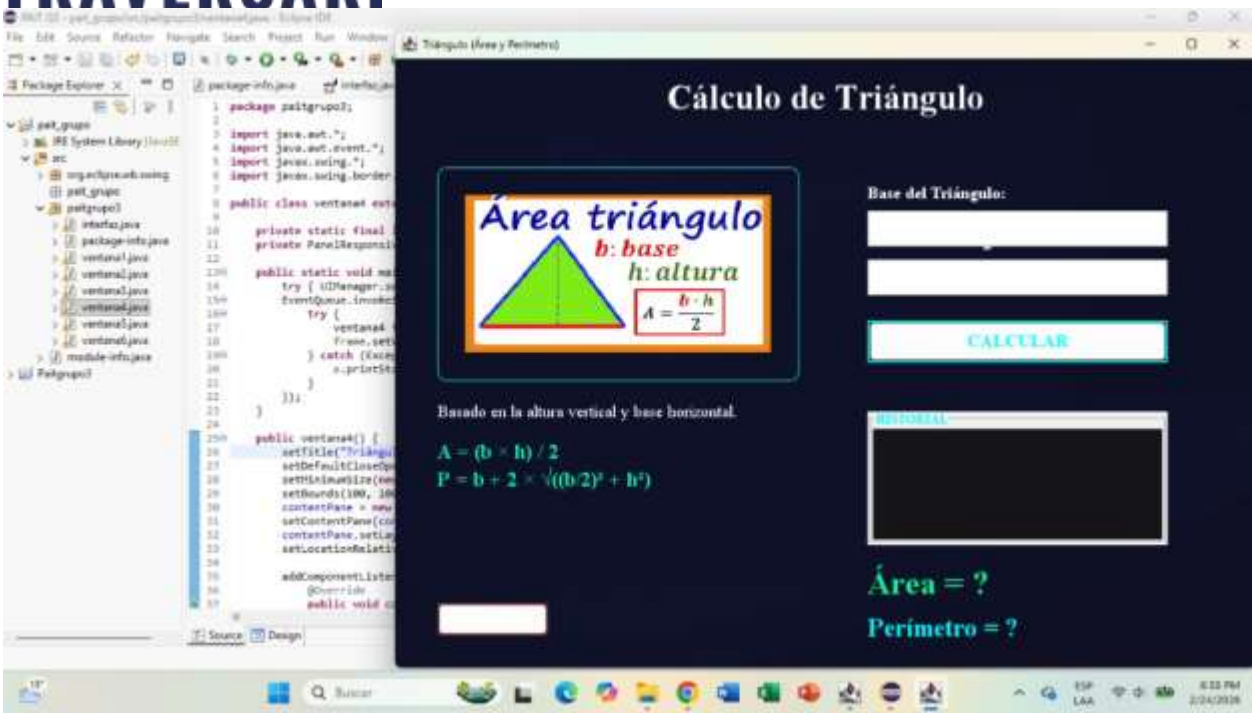
Source | Design

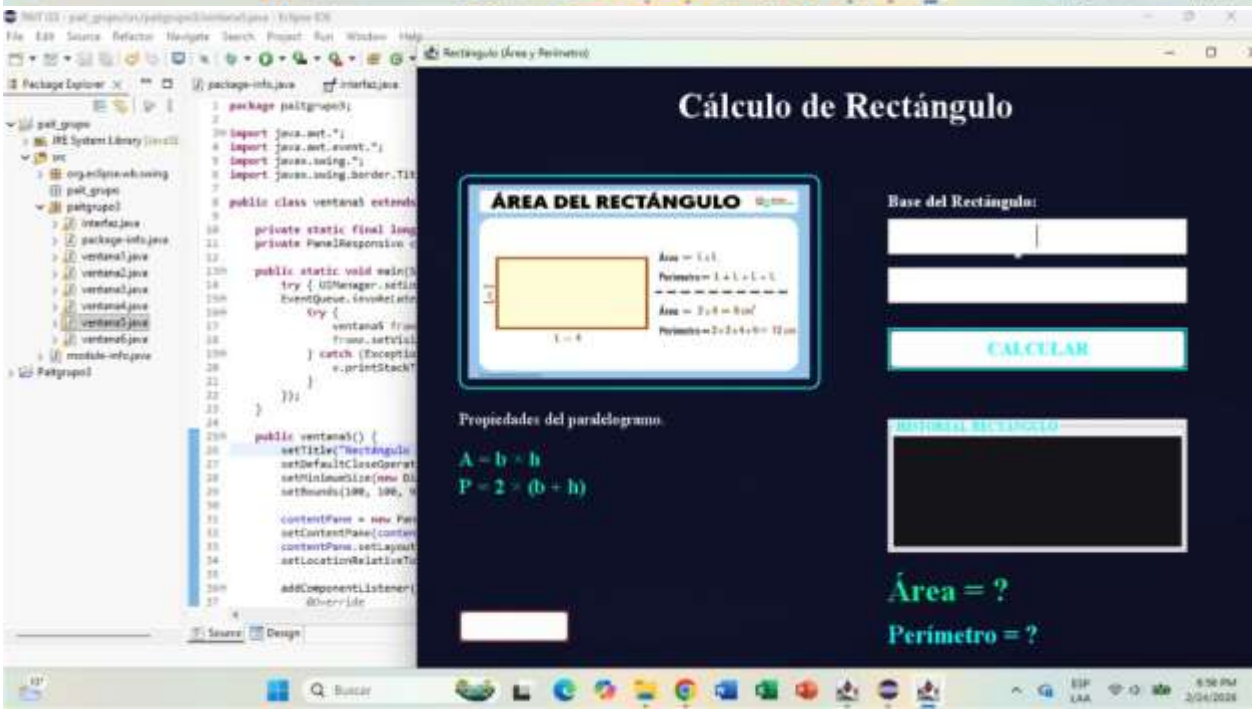


Posteriormente empezamos a darle forma a la interfaz gráfica con un estilo fácil de comprender para los estudiantes, con cada una de sus ventanas añadiendo formulas como guía para que sea aún más fácil de usar.






Triángulo (Área y Perímetro)
Cálculo de Triángulo
Área triángulo
b: base
h: altura
A = (b · h) / 2
Base del Triángulo:
CALCULAR
Basado en la altura vertical y base horizontal.
A = (b × h) / 2
P = b + 2 × √((b/2)² + h²)
Área = ?
Perímetro = ?


Rectángulo (Área y Perímetro)
Cálculo de Rectángulo
ÁREA DEL RECTÁNGULO
Base del Rectángulo:
CALCULAR
Propiedades del paralelogramo
A = b × h
P = 2 × (b + h)
Área = ?
Perímetro = ?

También nos permite tener un historial de los números que ya se ha calculado, en cada ventana se especifica que valores se deben ingresar a la calculadora para que no haya ningún tipo de error o confusión.

Se añadió un apartado gráfico para las ecuaciones por lo que sean más exactos los resultados y por ende más eficientes.

7. BIBLIOGRAFÍA

Alam, Ayaan. 2025. «What is JDK (Java Development Kit)?» https://intellipaat.com/blog/jdk-in-java/.

Arcila, Liliana. 2015. *if - else*.

Bernal, Luis. 2020. *Operadores Aritméticos*.

Bobkov, Alexey. 2014. *IDE Eclipse*.

Canelo, Miriam Martínez. 2025. «¿Qué es el pseudocódigo? | Profile». https://profile.es/blog/que-es-el-pseudocodigo/.

Carlos. 2022. *Trinomio*.

Casero, Alberto. 2023. «Compilación y ejecución en programación [Guía 2026]». https://keepcoding.io/blog/principios-de-compilacion-y-ejecucion/.

Código Root. 2022. *C. de Caracteres*.

EDEA. 2014. *F. Cuadratica*.

Evilnapsis. 2019. *T. Matriz*.

Gonzáles, Alfonso. 2014. *Funcion C*.

HACK A BOSS. 2025. *Bucle for*.

Lozada, Alexys. 2013. *A. Unidimensional*.

Lozada, Alexys. 2016. *Diferencia de cubos*.

Lozada, Alexys. 2022. *determinantes*.

Maldonado, Ramón. 2024. «Operadores aritméticos en programación: Conócelos». https://keepcoding.io/blog/operadores-aritmeticos-en-programacion/.

Maluenda, Raquel. 2024. «Qué es un algoritmo informático: características, tipos y ejemplos». https://profile.es/blog/que-es-un-algoritmo-informatico/.

Morales, Rafael. 2019. *pseudocódigo*.

ÓscarC. 2014. *Suma de cuadrados*.

Pérez, Carlos. 2022. *Operador Ternario*.

PruébaT. 2016. *M. Gráfico*.

¿Que es Java? | IBM. 2021. https://www.ibm.com/mx-es/think/topics/java.

¿Qué es Programación? | TripleTen Ecuador. s. f. Recuperado 2 de marzo de 2026. https://tripleten.ec/blog/que-es-programacion-4/.

¿Qué es un IDE? - Explicación de los entornos de desarrollo integrado - AWS. s. f. Recuperado 2 de marzo de 2026. https://aws.amazon.com/es/what-is/ide/.

Spanish GED 365. 2014. *T. Cuadrado P*.

Superprof. 2014. *Diagrama*.

Tecno-Comunicación. 2016. *M.Variable*.

Uriarte, Mario. 2021. *Modularización*.